

Machine Learning A (2026)

Home Assignment 1

Ludovico Maria Spitaleri, DMH249

Contents

1	Make Your Own (10 points)	2
2	Digits Classification with K Nearest Neighbors (40 points)	3
2.1	Task #1	3
2.1.1	K-NN Implementation	3
2.1.2	K-NN validation error for different values of n	4
2.1.3	K-NN variance of validation error over 5 data sets	6
2.1.4	Questions	6
2.2	Task #2 (optional, not for submission)	7
3	Regression (50 points)	7
3.1	Task 1 (5 points)	7
3.2	Task 2 (10 points)	8
3.3	Task 3 (10 points)	8
3.4	Task 4 (5 points)	10
3.5	Task 5 (10 points)	10
3.6	Task 6 (10 points)	11

1 Make Your Own (10 points)

1. The features I would include in the input space $\mathcal{X}$ would be:
 - The current GPA of the student $x_{\text{gpa}} \in [0, 1]$, which provides a good estimate of the general performance of the candidate.
 - The fields of study $x_{\text{field}} \in \{0, 1\}^k$, where k is the number of fields of study, since students with knowledge in related fields are more likely to perform well in the course.
 - n pairs of features, one for each existing course, containing whether the student attended it $x_{\text{attended}} \in \{0, 1\}$ and the final grade he obtained, with a value of $x_{\text{grade}} \in [0, 1]$ so that it is independent of any specific grading scale. This could help detect more specific correlations between a certain final grade in the ML course and other subjects than just including the GPA, that averages all subjects.

Combining all that, the final input space would be $\mathcal{X} = [0, 1] \times \{0, 1\}^k \times \{0, 1\}^n \times [0, 1]^n$.

2. The label space would be $\mathcal{Y} = [0, 1]$ such that the value can then easily be converted into any preferred grading scale.
3. Considering the label space $\mathcal{Y} = [0, 1]$, this would give an infinite number of options, making this problem a regression one. A squared error loss function $\ell(y, \hat{y}) = (y - \hat{y})^2$ makes the most sense since we can calculate the actual difference between the predicted and the real grade.
4. The distance measure used for this problem could be a simple Euclidean distance, given by the norm of the difference between the two vectors representing the student's features $d(x, x') = \|x - x'\|$.
5. The most important rule to evaluate a model is to avoid biased results caused by the reuse of data. So my evaluation process would consist of 3 phases:
 - (a) Take the first part of my dataset $\mathcal{S}_1$ to train the model and get a hypothesis class $\mathcal{H} = \{h_i : \mathcal{X} \rightarrow \mathcal{Y}\}$ for $i = 1, \dots, M$.
 - (b) Take the second part of my dataset $\mathcal{S}_2$ ($|\mathcal{S}_2| = N$) to select the hypothesis with the lowest average error.

$$r_i = \mathcal{R}_{\mathcal{S}_2}(h_i) = \frac{1}{N} \sum_{j=1}^N \ell(y_j, h_i(x_j)) \quad \forall i = 1, \dots, M$$
$$h_{i^*} \in \mathcal{H} \text{ with } i^* = \arg \min_i r_i$$

- (c) Take the third and final part of my dataset $\mathcal{S}_3$ to evaluate the selected hypothesis h_{i^*} by seeing the average error on new and unbiased data.
6. The main issue that I could expect is that the university could update, add or remove courses, making the model obsolete fast. A solution to that would be to train the model on a fixed schedule to be up to date with new data, but if a course was changed or added recently it would be difficult to have enough data to re-train the model.

2 Digits Classification with K Nearest Neighbors (40 points)

2.1 Task #1

2.1.1 K-NN Implementation

A *rng* parameter is added to have consistent results by providing the random generator for tie breaks. The signature of the function is still compatible with the requirements of the assignment since a default global generator is used by default.

```
def knn(training_points, training_labels, test_points, test_labels, rng=RNG)
:
    # rename to match the assignment notations
    X_train = training_points
    X_targ = test_points
    d, m = X_train.shape
    _, n = X_targ.shape

    assert X_targ.shape == (d, n)
    assert training_labels.shape == (m,)
    assert test_labels.shape == (n,)

    # compute the distances matrix
    XTX_train_diag = np.diag(X_train.T @ X_train).reshape(-1, 1)
    XTX_targ_diag = np.diag(X_targ.T @ X_targ).reshape(-1, 1)
    ones_n = np.ones((n, 1))
    ones_m = np.ones((m, 1))
    D = (
        (XTX_train_diag @ ones_n.T)
        - (2 * X_train.T @ X_targ)
        + (ones_m @ XTX_targ_diag.T)
    )
    assert D.shape == (m, n)

    # by mapping the labels {5, 6} to {-1, 1},
    # we can use the sum of the labels to determine the majority vote
    def labels_to_y(labels):
        return np.where(labels == 5, -1, 1)
```

```

Y_train = labels_to_y(training_labels)
Y_targ = labels_to_y(test_labels)

distances_sorted_idx = np.argsort(D, axis=0)
Y_train_sorted = Y_train[distances_sorted_idx]
Y_train_running_sum = np.cumsum(Y_train_sorted, axis=0)

# since the labels are {-1, 1}, the sign of the running sum gives the
majority vote,
# and we can break ties by randomly choosing a label
tie_break = rng.choice([-1, 1], size=Y_train_running_sum.shape)
prediction = np.where(
    Y_train_running_sum == 0, tie_break, np.sign(Y_train_running_sum)
)

# compare to the true labels and compute the accuracy for each K
correct = prediction != Y_targ.T
error_per_K = np.mean(correct, axis=1)
return error_per_K

```

2.1.2 K-NN validation error for different values of n

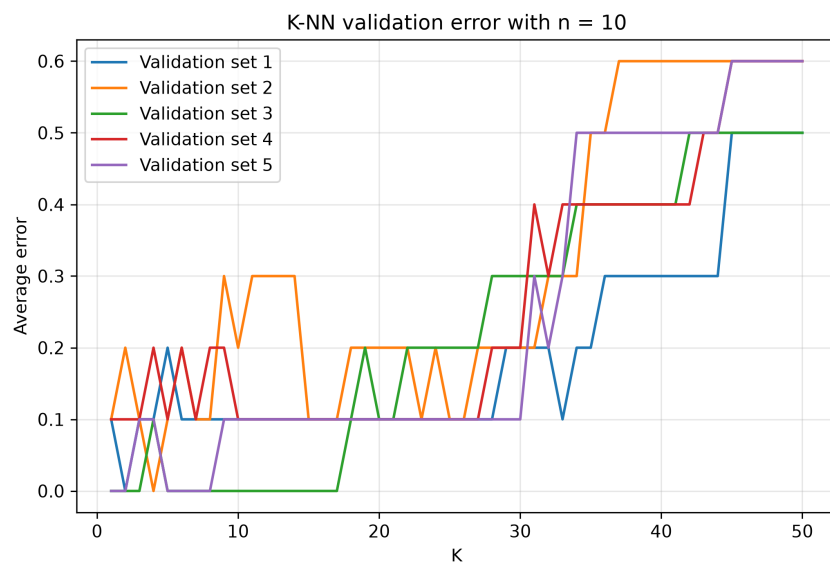


Figure 1: K-NN validation error with $n = 10$

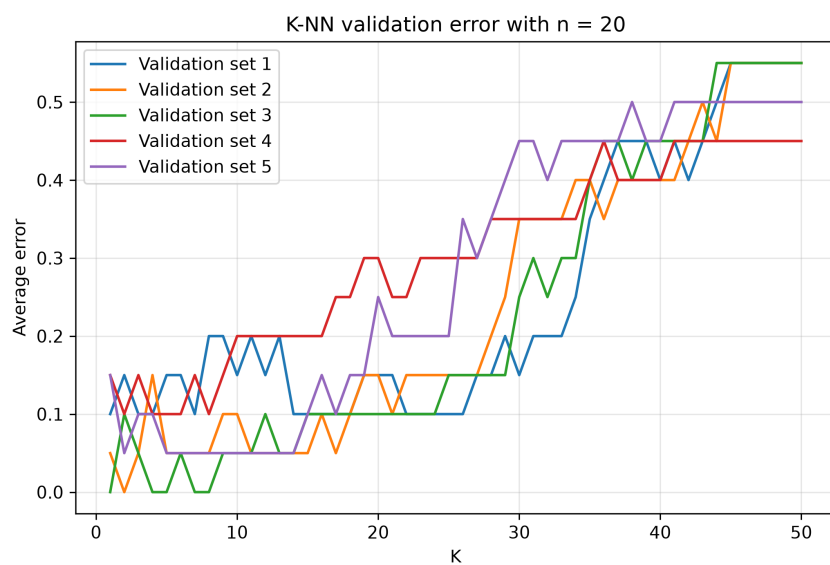


Figure 2: K-NN validation error with $n = 20$

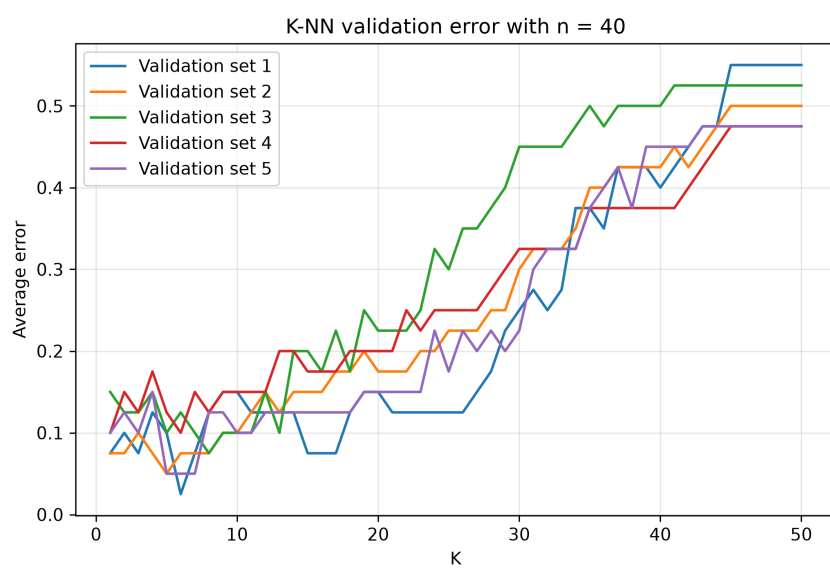


Figure 3: K-NN validation error with $n = 40$

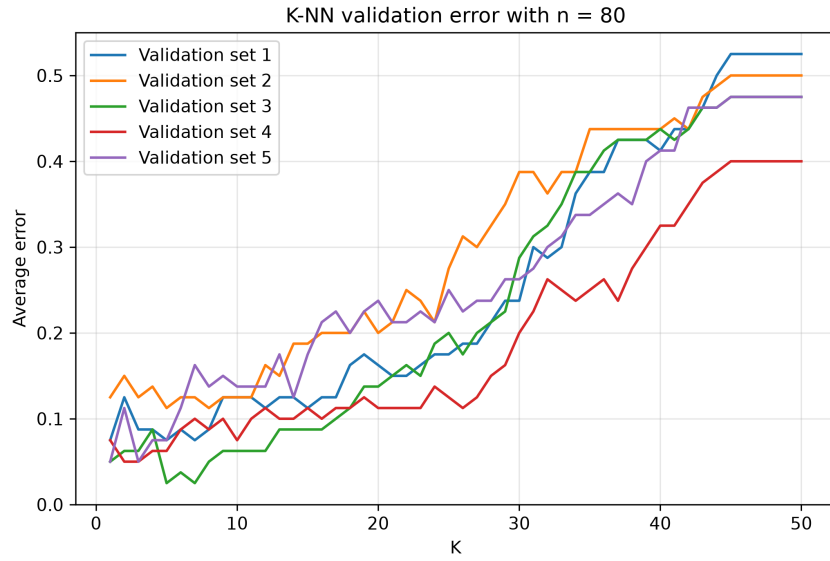


Figure 4: K-NN validation error with $n = 80$

2.1.3 K-NN variance of validation error over 5 data sets

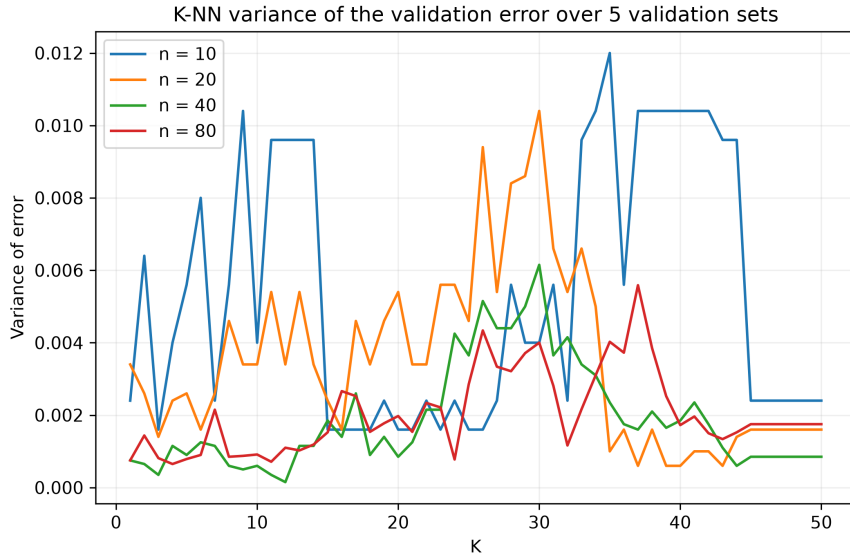


Figure 5: K-NN validation error variance

2.1.4 Questions

- As the validation set size n grows, the validation error computed on each of the five sets becomes a more reliable estimate of the true risk $\mathcal{R}(h)$. The variance of the

validation error shrinks as the sample size increases, consistent with $\mathcal{R}_S(h)$ being an unbiased estimator of $\mathcal{R}(h)$.

- For small values of K , the classifier tends to suffer of overfitting problems because of non-representative training points. As K increases, noise is averaged out and generalization improves. If K becomes too large, locality principles are lost, and the model predicts the majority class in the entire dataset instead of only the neighbors of the training point. Therefore, the optimal value of K lies in between 1 and m .

2.2 Task #2 (optional, not for submission)

3 Regression (50 points)

3.1 Task 1 (5 points)

```
def linear_regression(X, y):
    X_bias = np.hstack((X, np.ones((X.shape[0], 1))))
    X_dagger = np.linalg.pinv(X_bias)
    w_bias = X_dagger @ y
    w, b = w_bias[:-1], w_bias[-1]
    f = lambda X: (X @ w) + b # since we operate on the whole matrix, w.T @
    x becomes X @ w
    return w, b, f
```

A function to calculate the mean squared error is also defined to verify the results

```
def mean_squared_error(y, y_hat):
    return np.mean((y - y_hat) ** 2)
```

Usage:

```
w, b, h = linear_regression(X, y)
y_hat = h(X)
mse = mean_squared_error(y, y_hat)
```

Results:

$$\begin{aligned} a &= 1.558 \\ b &= -1.452 \\ MSE &= 24.801 \end{aligned}$$

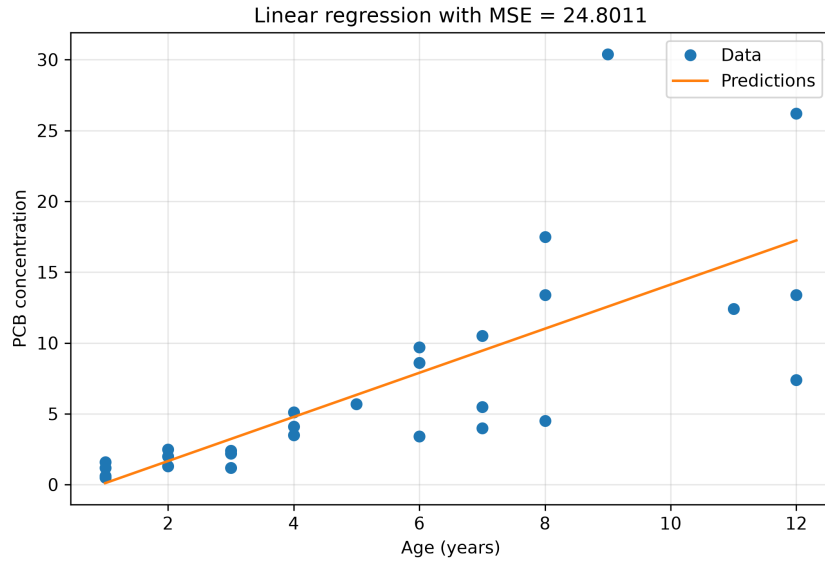


Figure 6: Linear regression

3.2 Task 2 (10 points)

```
y_ln = np.log(y)
w_ln, b_ln, h_ln = linear_regression(X, y_ln)
y_hat_exp = np.exp(h_ln(X))
mse_exp = mean_squared_error(y, y_hat_exp)
```

Results:

$$a = 0.259$$

$$b = 0.031$$

$$MSE = 34.836$$

3.3 Task 3 (10 points)

We want to show that

$$\arg \min_{a,b} \sum_{i=1}^N (y_i - h(x_i))^2 \neq \arg \min_{a,b} \sum_{i=1}^N (\ln y_i - (ax_i + b))^2,$$

where $h(x) = \exp(ax + b)$.

Lets consider a data set with two points, $S = \{(x_1, y_1), (x_2, y_2)\}$, where:

$$x_1 = x_2 = 0$$

$$y_1 = e^{-1}, y_2 = e^1.$$

Since $x_1 = x_2 = 0$, the model $h'(x) = ax + b$ evaluates to the same value regardless of a . So in both fitting problems we are choosing a single constant prediction of b for the two points.

First we minimize on the original scale

$$\begin{aligned}\sum_{i=1}^N (y_i - h(x_i))^2 &= (y_1 - \exp(b))^2 + (y_2 - \exp(b))^2 \\ \frac{d}{db} &= -2(y_1 - \exp(b)) - 2(y_2 - \exp(b)) = 0 \\ \implies \exp(b) &= \frac{e^{-1} + e^1}{2} \approx 1.543\end{aligned}$$

Then we minimize on the logarithmic scale

$$\begin{aligned}\sum_{i=1}^N (\ln y_i - (ax_i + b))^2 &= (\ln y_1 - b)^2 + (\ln y_2 - b)^2 \\ \frac{d}{db} &= -2(\ln y_1 - b) - 2(\ln y_2 - b) = 0 \\ \implies b &= \frac{-1 + 1}{2} = 0\end{aligned}$$

Conclusion. The optimal constant fit on the original scale is $b \approx 1.543$, while the optimal constant fit obtained via the log-transformed objective is $h(0) = 1$. Since $1.543 \neq 1$, the parameters minimizing $\sum_i (y_i - h(x_i))^2$ differ from those minimizing $\sum_i (\ln y_i - (ax_i + b))^2$.

3.4 Task 4 (5 points)



Figure 7: Non-linear $h(x) = \exp(ax + b)$ regression

3.5 Task 5 (10 points)

```
def r2_score(y, y_hat):  
    y_bar = np.mean(y)  
    return 1 - (np.sum((y - y_hat) ** 2) / np.sum((y - y_bar) ** 2))  
  
r2_exp = r2_score(y, y_hat_exp)
```

The coefficient of determination R^2 indicates how well the model performs compared to its average error.

- $R^2 = 1$: the numerator is 0, so the model fits the data perfectly with no error.
- $R^2 = 0$: the numerator and denominator are equal, so the model predicts exactly like always picking the average label.
- $R^2 < 0$: the numerator is bigger than the denominator, so the model performs worse than always picking the average.

The coefficient of determination of the non-linear model $h(x) = \exp(ax + b)$ is $R^2 = 0.357$.

3.6 Task 6 (10 points)

```
y_ln = np.log(y)
X_sqrt = np.sqrt(X)
w_sqrt_ln, b_sqrt_ln, h_sqrt_ln = linear_regression(X_sqrt, y_ln)
y_hat_sqrt_exp = np.exp(h_sqrt_ln(X_sqrt))

mse_sqrt_exp = mean_squared_error(y, y_hat_sqrt_exp)
r2_sqrt_exp = r2_score(y, y_hat_sqrt_exp)
```

Results:

$$a = 1.199$$

$$b = -1.194$$

$$MSE = 28.084$$

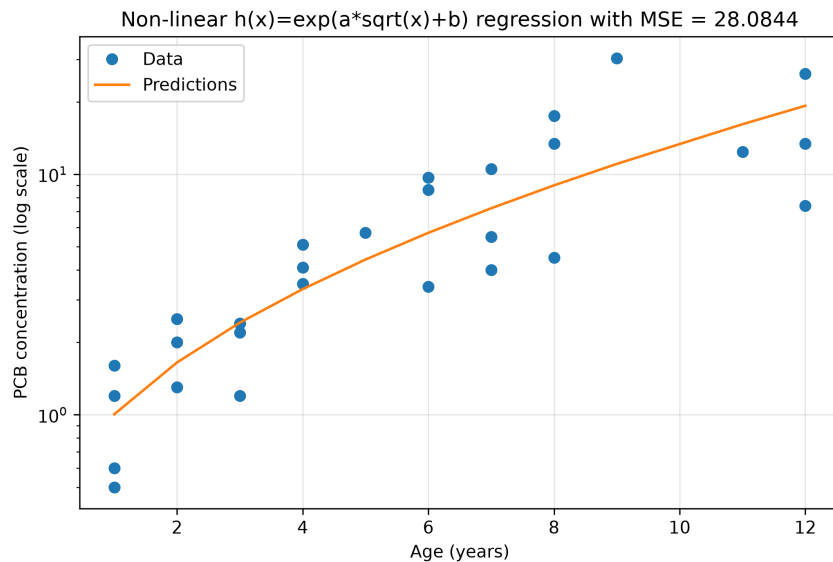


Figure 8: Non-linear $h(x) = \exp(a\sqrt{x} + b)$ regression

The coefficient of determination of the non-linear model $h(x) = \exp(a\sqrt{x} + b)$ is $R^2 = 0.482$. The improved R^2 score demonstrates that the underlying relationship is closer to a square-root dependency than a strictly linear one. Because this new feature space captures the data's trajectory more accurately, the resulting model fits the data better, reducing the mean squared error.